

```

• x2 = Dense(25, activation= 'relu')(input _ metadata)
• x2 = Dropout(0.3)(x2)
•
• con = concatenate(inputs = [x1,x2] ) # merge in metadata
• x3 = Dense(50)(con)
• x3 = Dropout(0.3)(x3)
• output = Dense(1, activation='sigmoid')(x3)
• meta _ n _ net = Model(inputs=[input _ dat, input _ meta-
data], outputs=output)
• meta _ n _ net.compile(optimizer='sgd', loss='binary _ cros-
sentropy', metrics=['accuracy'])
• Obtaining average values of the models can be done from
the following code in Keras
• def get _ model():
•     inputs = keras.Input(shape=(128,))
•     outputs = layers.Dense(1, activation='sigmoid')(inputs)
•     return keras.Model(inputs, outputs)
•
• model1 = get _ model()
• model2 = get _ model()
• model3 = get _ model()
• inputs = keras.Input(shape=(128,))
• y1 = model1(inputs)
• y2 = model2(inputs)
• y3 = model3(inputs)
• outputs = layers.average([y1, y2, y3])
• ensemble _ model = keras.Model(inputs=inputs,
outputs=outputs)
• feat _ extraction _ model = keras.Model(inputs=vgg19.input,
outputs=features _ list)
• img = np.random.random((1, 224, 224, 3)).astype('float32')
• extracted _ features = feat _ extraction _ model(img)

```

FUNCTIONAL API TESTING

Once the model has been set and good to go, it's now time to move on to API Testing. Testing is usually a good practice to help validate the results obtained from the analysis and ensure that it can hold well against any kind of input.

The most common protocol that's used in API testing is HTTP which when combined with the REST architecture connects the code to the interface for users. Any good API functional test begins with checking out the